Atividade 10 – Capacidade de Canais de Comunicação

Nome: Matheus Milanezi

Disciplina: Redes de Computadores: Internetworking, Roteamento e Transmissão

Linguagem: Python 3.14 (arquivo único `capacidade.py`; matplotlib 3.11 para os gráficos)

Execução: 31/10 e 01/11/2026, no PowerShell (Windows 10), dentro da pasta da atividade

1. Objetivo

Implementar os modelos de capacidade de canal de Nyquist e de Shannon, validar a implementação e usá-la em quatro experimentos que mostram o efeito da largura de banda (B), do número de níveis de sinalização (N) e da relação sinal-ruído (SNR) sobre a capacidade (C).

2. Implementação

Modelos

Modelo	Fórmula	Tipo de canal	Parâmetros
Nyquist	$C = 2B \cdot \log_2(N)$	ideal, sem ruído	B em Hz; N = níveis de sinalização
Shannon	$C = B \cdot \log_2(1 + SNR)$	com ruído	B em Hz; SNR em escala linear

A SNR em decibéis é convertida para a escala linear por SNR = 10^(SNR_dB/10), e o caminho inverso é $SNR_{dB} = 10 \cdot \log_{10}(SNR)$.

Os dois modelos estão ligados. A fórmula de Shannon é obtida a partir da de Nyquist: o ruído limita o número de níveis que o receptor consegue distinguir a **$N_{max} = \sqrt{(1 + SNR)}$**, e, substituindo esse valor em Nyquist, $2B \cdot \log_2\sqrt{(1 + SNR)} = B \cdot \log_2(1 + SNR)$. O programa usa essa relação para mostrar N_{max} junto com o resultado de Shannon e confere a identidade no modo `validar`.

Estrutura do programa

Função	O que faz
`nyquist(B, N)`	Calcula $C = 2B \cdot \log_2(N)$, em bit/s
`shannon(B, snr)`	Calcula $C = B \cdot \log_2(1 + SNR)$, em bit/s
`db_para_linear`, `linear_para_db`	Convertem a SNR entre dB e a escala linear
`n_max(snr)`	Calcula o número máximo de níveis distinguíveis, $\sqrt{(1 + SNR)}$
`ler_frequencia`, `ler_niveis`, `ler_snr`, `ler_snr_db`	Leem e validam os parâmetros: B > 0 (aceita os sufixos k, M e G), N inteiro ≥ 2, SNR linear ≥ 0
`taxa`, `frequencia`, `snr_texto`	Mostram os resultados com unidades: bit/s, kbit/s ou Mbit/s; Hz ou kHz; SNR linear e em dB
`experimento_1` a `experimento_4`	Calculam as tabelas, imprimem observações calculadas a partir dos resultados e geram os gráficos
`modo_validar`	Confere o programa com os exemplos da aula 12 e com verificações internas

Formas de execução:

```
python capacidade.py                                        os 4 experimentos (tabelas + gráficos)
python capacidade.py nyquist  --B 3k --N 8                  capacidade de Nyquist
python capacidade.py shannon  --B 3k --snr 1000             Shannon com SNR linear
python capacidade.py shannon  --B 3k --snr-db 30            Shannon com SNR em dB
python capacidade.py comparar --B 3k --snr-db 30 --N 32     Nyquist x Shannon (+ N_max)
python capacidade.py validar                                confere com os exemplos da aula 12
```

Os resultados são mostrados no padrão brasileiro (11.111,11) e sempre com unidade. Os gráficos são gerados com matplotlib e salvos na pasta `graficos/`, e a saída dos experimentos também é gravada em `saida_experimentos.txt`. O código-fonte completo está no **Apêndice A**.

Modo parâmetro

O exemplo da aula (B = 20 Hz, N = 4) foi reproduzido: o canal transmite 2B = 40 símbolos por segundo, cada um com $\log_2 4 = 2$ bits, o que dá 80 bit/s (Figura 1).

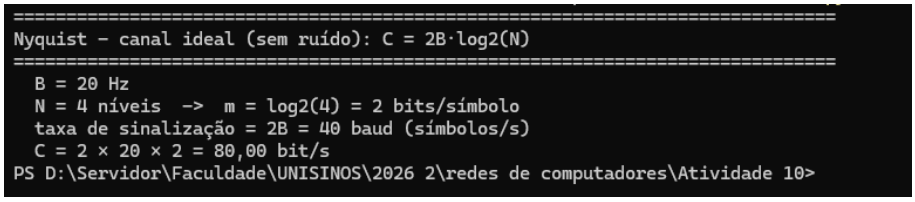


Figura 1 – python capacidade.py nyquist --B 20 --N 4 .

O cálculo de Shannon foi executado duas vezes para o mesmo canal, uma com a SNR em dB e outra em escala linear. Os dois resultados são idênticos (29.901,68 bit/s), o que confere a conversão de dB (Figuras 2 e 3).

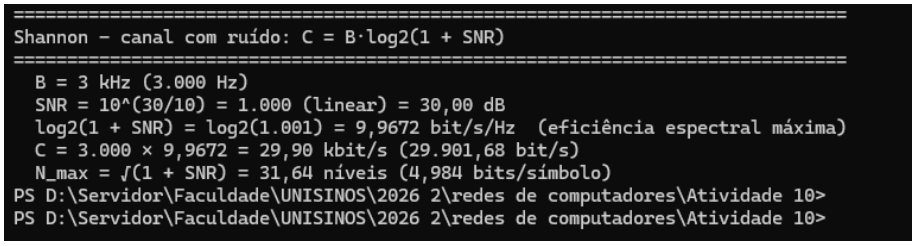


Figura 2 – python capacidade.py shannon --B 3k --snr-db 30.

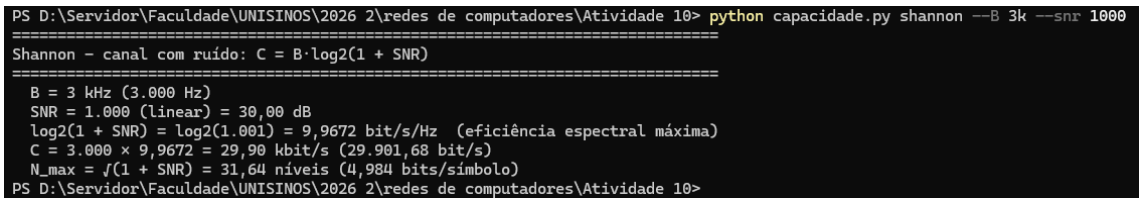


Figura 3 – python capacidade.py shannon --B 3k --snr 1000.

O modo `comparar` coloca os dois modelos lado a lado para os mesmos B e SNR (Figura 4), e um valor de N que não é potência de 2 também é aceito: com N = 3, cada símbolo carrega $\log_2 3 = 1{,}585$ bit, o que só faz sentido como limite teórico (Figura 5).

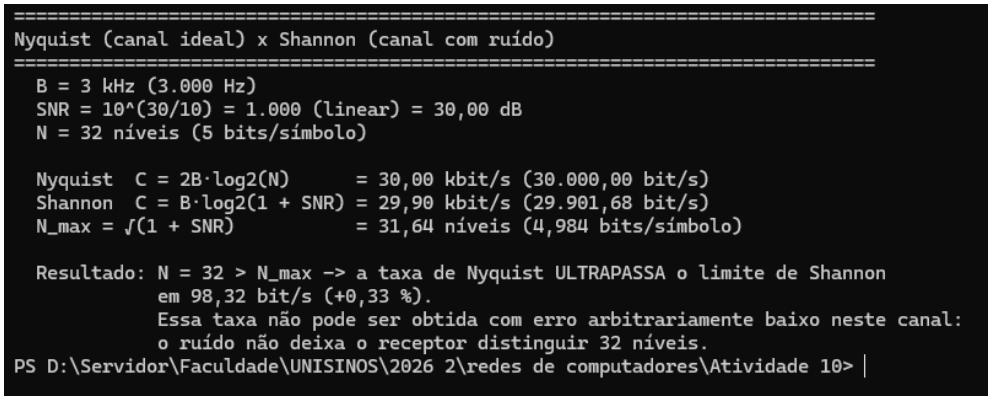


Figura 4 – python capacidade.py comparar --B 3k --snr-db 30 --N 32.

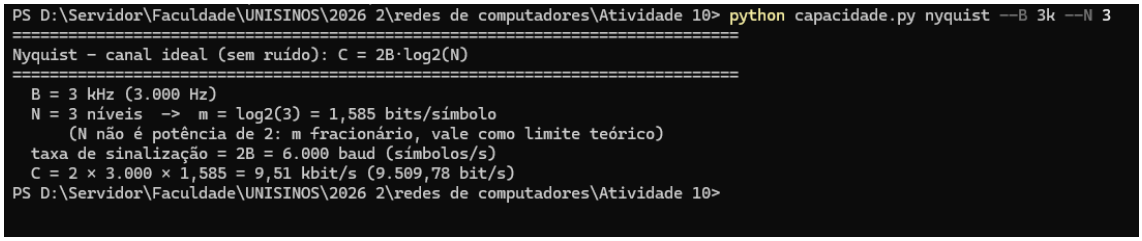


Figura 5 – python capacidade.py nyquist --B 3k --N 3.

Validação da entrada

Valores sem sentido físico são recusados com uma mensagem de erro, em vez de gerar um resultado:

- **N = 1** (Figura 6): $\log_2 1 = 0$, ou seja, um sinal com um único nível nunca muda e não transporta informação. O programa exige N inteiro ≥ 2.
- **SNR = 1.000** (Figura 7): o próprio enunciado escreve a SNR com ponto de milhar (1.000 = mil), mas em Python "1.000" é lido como 1,0, e o resultado seria $\log_2(2) = 1$ bit/s/Hz em vez de 9,967. Como as duas leituras são possíveis, o programa recusa a entrada e pede o número sem separador.

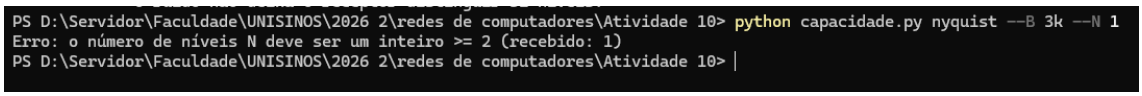


Figura 6 – Entrada inválida: N = 1.

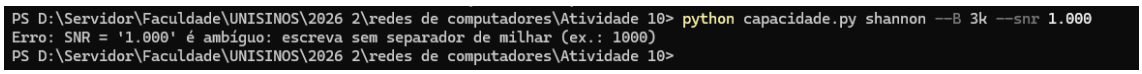


Figura 7 – Entrada ambígua: SNR = 1.000.

Também são recusados B ≤ 0, SNR linear negativa, N fracionário e textos que não são números (essas entradas são testadas no modo `validar`).

3. Validação da implementação

Antes dos experimentos, o modo `validar` comparou o programa com os exemplos numéricos da aula 12 e com verificações que não dependem dos valores do enunciado. As 9 verificações passaram (Figura 8):

Verificação	Resultado
Slide 14: Nyquist com B = 20 Hz e N = 4	80,00 bit/s (esperado 80 bit/s)
Slide 23: 30 dB em escala linear	10^(30/10) = 1.000
Slide 24: Shannon com B = 3.000 Hz e SNR = 1.000	29.901,68 bit/s, igual ao cálculo pelo caminho do slide ($\log_{10}$ 1001 / $\log_{10}$ 2)
Slide 24: valor do slide, com $\log_{10}$ 1001 ≈ 3 e $\log_{10}$ 2 ≈ 0,3	3.000 × 3/0,3 = 30.000 bit/s (diferença de 98,32 bit/s, 0,33 %)
Conversão dB → linear → dB	5/5 valores voltam ao original
Identidade $2B\cdot\log_2\sqrt{(1 + SNR)} = B\cdot\log_2(1 + SNR)$	12/12 combinações de B e SNR
Nyquist com N = 2^m é exatamente 2B·m	18/18 combinações
Extremos: SNR = 0 → C = 0; SNR = 0 dB → C = B	0 bit/s e 5.000 bit/s (com B = 5 kHz)
Entradas inválidas são rejeitadas	7/7

```
PS D:\Servidor\Faculdade\UNISINOS\2026 2\redes de computadores\Atividade 10> python capacidade.py validar
================================================================================
VALIDAÇÃO - exemplos da aula 12 e conferências internas
================================================================================
  [OK] Aula 12, slide 14: Nyquist com B = 20 Hz e N = 4
       obtido 80,00 bit/s, esperado 80 bit/s
  [OK] Aula 12, slide 23: 30 dB em escala linear
       obtido 10^(30/10) = 1.000, esperado 1.000
  [OK] Aula 12, slide 24: Shannon com B = 3.000 Hz e SNR = 1.000 (valor exato)
       obtido 29,90 kbit/s (29.901,68 bit/s)
       pelo caminho do slide: 3.000 × log10(1.001)/log10(2) = 29.901,68 bit/s
  [OK] Aula 12, slide 24: valor do slide com os arredondamentos log10(1.001) ≈ 3, log10(2) ≈ 0,3
       3.000 × 3/0,3 = 30.000 bit/s; diferença para o exato = 98,32 bit/s (0,33 %)
  [OK] Conversão dB -> linear -> dB volta ao valor original
       5/5 valores (0, 3, 10, 30, 47 dB)
  [OK] Identidade da aula: 2B·log2(√(1 + SNR)) = B·log2(1 + SNR)
       12/12 combinações de B e SNR
  [OK] Nyquist com N = 2^m é exatamente 2B·m
       18/18 combinações (m = 1 a 6)
  [OK] Shannon nos extremos: SNR = 0 -> C = 0 e SNR = 0 dB -> C = B
       C(SNR = 0) = 0,00 bit/s; C(0 dB) = 5.000,00 bit/s com B = 5.000 Hz
  [OK] Entradas inválidas são rejeitadas
       7/7: N = 1, N = 2.5, B = -5, B = 0, SNR = -3, SNR = '1.000' (ambíguo), SNR = 'abc'

Resultado: 9/9 verificações OK
```

Figura 8 – python capacidade.py validar.

O exemplo do slide 24 merece atenção. O slide chega a **30.000 bit/s** porque arredonda $\log_2(1001)$ para 10, mas o valor exato é **29.901,68 bit/s**. A diferença é pequena (0,33 %), mas muda a conclusão do Experimento 4, que usa exatamente esse canal (B = 3 kHz e 30 dB).

4. Experimento 1 – Capacidade de Nyquist

B = 1, 5 e 10 kHz e N = 2, 4, 8 e 16 níveis, com $C = 2B\cdot\log_2(N)$. Capacidade em kbit/s:

B \ N	2 (1 bit)	4 (2 bits)	8 (3 bits)	16 (4 bits)
1 kHz	2,00	4,00	6,00	8,00
5 kHz	10,00	20,00	30,00	40,00
10 kHz	20,00	40,00	60,00	80,00

```
================================================================================
EXPERIMENTO 1 - Capacidade de Nyquist (canal ideal): C = 2B·log2(N)
================================================================================
Capacidade C em kbit/s:

  N (níveis)                      2        4        8       16
  m = log2 N (bits/símbolo)       1        2        3        4
  ------------------------------------------------------------------------
  B = 1 kHz                    2,00     4,00     6,00     8,00
  B = 5 kHz                   10,00    20,00    30,00    40,00
  B = 10 kHz                  20,00    40,00    60,00    80,00

Observações calculadas:
  - B x 10 (1 -> 10 kHz): C x 10,00 para todo N -> C é proporcional a B
  - B = 1 kHz: cada vez que N dobra (+1 bit/símbolo), C soma 2,00 kbit/s (= 2B)
  - B = 5 kHz: cada vez que N dobra (+1 bit/símbolo), C soma 10,00 kbit/s (= 2B)
  - B = 10 kHz: cada vez que N dobra (+1 bit/símbolo), C soma 20,00 kbit/s (= 2B)
  - N x 8 (2 -> 16 níveis) multiplica C por apenas 4,00 (= log2 16 / log2 2)
  gráfico salvo em: graficos\exp1_nyquist.png
```

Figura 9 – Saída do Experimento 1 (python capacidade.py).

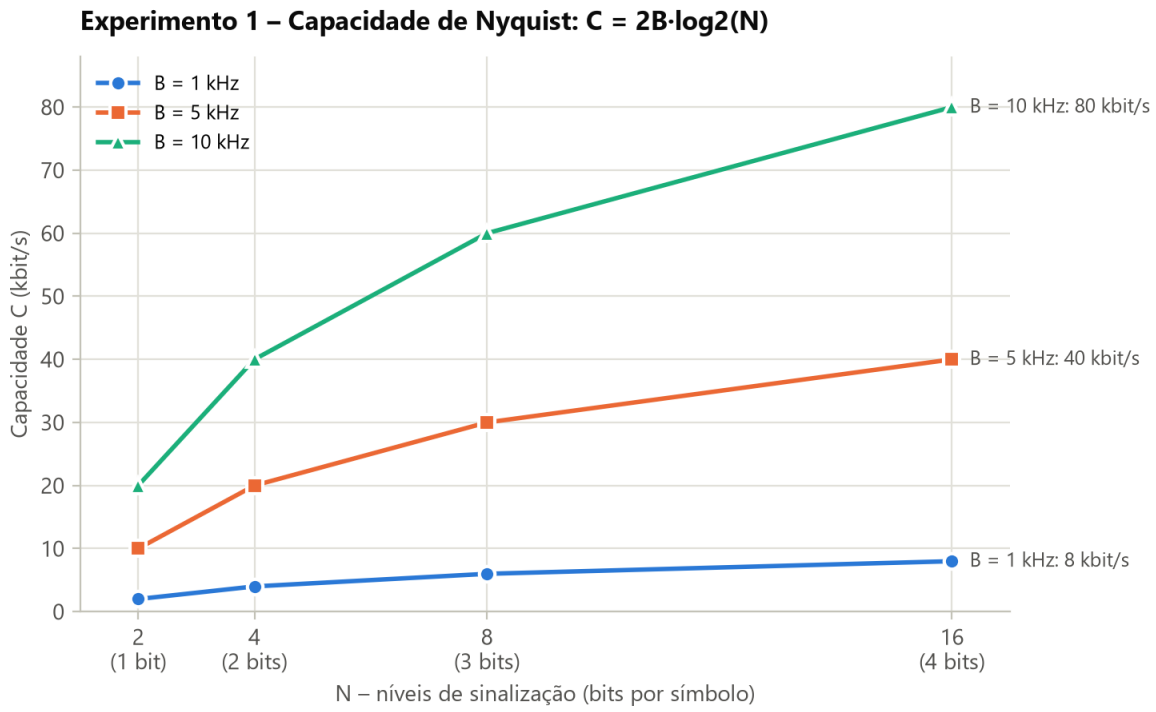


Figura 10 – Experimento 1: C × N, uma curva para cada largura de banda.

Como a largura de banda influencia a capacidade?

A capacidade é diretamente proporcional a B. De 1 para 10 kHz, C foi multiplicada por exatamente 10 para todos os valores de N (por exemplo, de 8 para 80 kbit/s com N = 16). Pelo teorema de Nyquist, um canal com largura de banda B transmite no máximo 2B símbolos por segundo, e a capacidade é essa taxa de símbolos vezes os bits por símbolo.

Qual é o impacto de aumentar o número de bits por símbolo?

Cada bit a mais por símbolo soma 2B bit/s à capacidade: +2, +10 e +20 kbit/s para B = 1, 5 e 10 kHz, constante ao longo de toda a tabela. Mas cada bit a mais exige **dobrar** o número de níveis. Por isso a capacidade cresce de forma linear com os bits por símbolo e só logarítmica com N: multiplicar N por

8 (de 2 para 16 níveis) multiplicou C por apenas 4. No gráfico, as curvas ficam cada vez mais "achatadas" à medida que N aumenta.

Por que Nyquist, isoladamente, não representa um canal com ruído?

Porque a fórmula não tem nenhum termo de ruído: para B fixo, basta aumentar N para a capacidade crescer sem limite. A aula 12 aponta exatamente isso: Nyquist não considerou um limite para N. Num canal real, com a amplitude máxima do sinal fixa, mais níveis significam níveis mais próximos uns dos outros, e quando a distância entre eles fica menor que o ruído, o receptor não consegue mais distingui-los. O Experimento 4 mostra esse limite em números.

5. Experimento 2 – Capacidade de Shannon

SNR = 1.000, 5.000, 10.000 e 50.000 (escala linear) e B de 1 a 10 kHz, com $C = B \cdot \log_2(1 + SNR)$. Capacidade em kbit/s:

B \ SNR	**1.000 (30,0 dB)**	**5.000 (37,0 dB)**	**10.000 (40,0 dB)**	**50.000 (47,0 dB)**
$\log_2(1 + SNR)$	9,967 bit/s/Hz	12,288 bit/s/Hz	13,288 bit/s/Hz	15,610 bit/s/Hz
1 kHz	**9,97** (menor)	12,29	13,29	15,61
2 kHz	19,93	24,58	26,58	31,22
3 kHz	29,90	36,86	39,86	46,83
4 kHz	39,87	49,15	53,15	62,44
5 kHz	49,84	61,44	66,44	78,05
6 kHz	59,80	73,73	79,73	93,66
7 kHz	69,77	86,02	93,01	109,27
8 kHz	79,74	98,30	106,30	124,88
9 kHz	89,71	110,59	119,59	140,49
10 kHz	99,67	122,88	132,88	**156,10** (maior)

- **Maior capacidade:** 156,10 kbit/s (156.096,69 bit/s), com B = 10 kHz e SNR = 50.000.
- **Menor capacidade:** 9,97 kbit/s (9.967,23 bit/s), com B = 1 kHz e SNR = 1.000.

A razão entre as duas é 15,66. Desse fator, o aumento de B (×10) contribuiu com ×10, e o aumento de SNR (×50) só com ×1,566.

```
================================================================================
EXPERIMENTO 2 - Capacidade de Shannon: C = B·log2(1 + SNR)
================================================================================
Capacidade C em kbit/s:

  SNR (linear)                1.000       5.000      10.000      50.000
  SNR (dB)                    30,00       36,99       40,00       46,99
  log2(1 + SNR) (bit/s/Hz)    9,967      12,288      13,288      15,610
  --------------------------------------------------------------------------
  B = 1 kHz                    9,97       12,29       13,29       15,61
  B = 2 kHz                   19,93       24,58       26,58       31,22
  B = 3 kHz                   29,90       36,86       39,86       46,83
  B = 4 kHz                   39,87       49,15       53,15       62,44
  B = 5 kHz                   49,84       61,44       66,44       78,05
  B = 6 kHz                   59,80       73,73       79,73       93,66
  B = 7 kHz                   69,77       86,02       93,01      109,27
  B = 8 kHz                   79,74       98,30      106,30      124,88
  B = 9 kHz                   89,71      110,59      119,59      140,49
  B = 10 kHz                  99,67      122,88      132,88      156,10

Maior capacidade: 156,10 kbit/s (156.096,69 bit/s)  (B = 10 kHz, SNR = 50.000)
Menor capacidade: 9,97 kbit/s (9.967,23 bit/s)  (B = 1 kHz, SNR = 1.000)
Maior / menor = 15,66

Observações calculadas:
  - Para cada SNR, C/B é constante: C(10 kHz) / C(1 kHz) = 10,00 -> C cresce linearmente com B
  - SNR x 5 (1.000 -> 5.000): C x 1,2328 para todo B; a eficiência sobe só 2,3208 bit/s/Hz
  - SNR x 5 (10.000 -> 50.000): C x 1,1747 para todo B; a eficiência sobe só 2,3218 bit/s/Hz
  - SNR x 50 (1.000 -> 50.000): C x 1,5661 para todo B; a eficiência sobe só 5,6424 bit/s/Hz
  - Para C x 5 a partir de SNR = 1.000 seria preciso (1 + SNR)^5 - 1 = 1,005 x 10^15 (150,0 dB)
  gráfico salvo em: graficos\exp2_shannon.png
```

Figura 11 – Saída do Experimento 2.

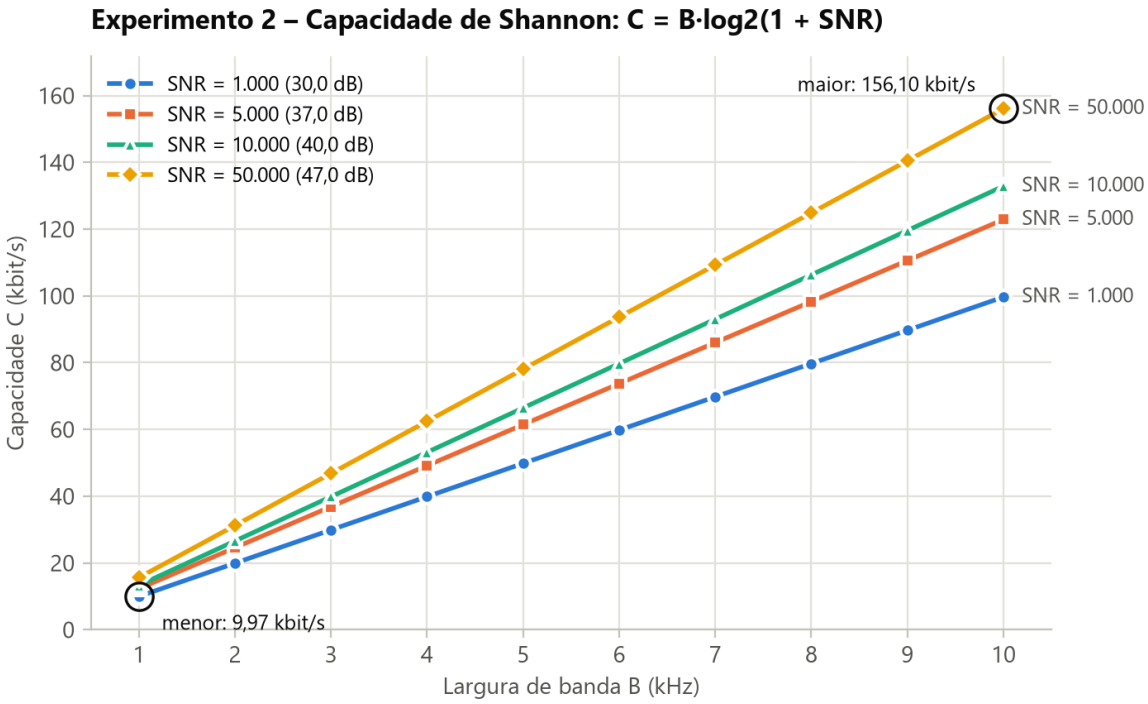


Figura 12 – Experimento 2: C × B, uma curva para cada SNR.

Como a capacidade varia com B?

Linearmente. Para cada SNR, a razão C/B é constante e igual a $\log_2(1 + SNR)$, a eficiência espectral máxima do canal (9,967 bit/s/Hz para SNR = 1.000, por exemplo). Por isso as quatro curvas do gráfico são retas que partem da origem, e C(10 kHz)/C(1 kHz) = 10 para todas as SNRs.

Qual é o impacto do aumento da SNR?

A SNR aumenta a inclinação das retas, mas pouco. Multiplicar a SNR por 50 (de 1.000 para 50.000) multiplicou a capacidade por apenas 1,566, enquanto multiplicar B por 10 multiplicou a capacidade por 10. A SNR entra na fórmula dentro do logaritmo, e B entra fora dele.

Multiplicar a SNR por cinco multiplica a capacidade por cinco?

Não. De 1.000 para 5.000, a capacidade foi multiplicada por **1,2328**, e de 10.000 para 50.000 por **1,1747**, para qualquer B. Multiplicar a SNR por 5 **soma** cerca de $\log_2 5 \approx 2{,}32$ bit/s/Hz à eficiência (o programa mediu 2,3208 e 2,3218), em vez de multiplicar o resultado. Para que a capacidade realmente quintuplicasse a partir de SNR = 1.000, seria preciso $(1 + SNR)^5 - 1 \approx 1{,}005 \times 10^{15}$, ou seja, 150 dB, um valor fora de qualquer realidade. Já multiplicar B por 5 multiplica a capacidade por 5.

6. Experimento 3 – Influência do ruído

B = 5 kHz e SNR de 0 a 40 dB, em passos de 5 dB, com SNR = 10^(SNR_dB/10) e $C = B \cdot \log_2(1 + SNR)$:

SNR (dB)	SNR (linear)	$\log_2(1 + SNR)$	C (kbit/s)	ΔC em relação a 5 dB antes
0	1,00	1,000	5,00	–
5	3,16	2,057	10,29	+5,29
10	10,00	3,459	17,30	+7,01
15	31,62	5,028	25,14	+7,84
20	100,00	6,658	33,29	+8,15
25	316,23	8,309	41,55	+8,26
30	1.000,00	9,967	49,84	+8,29
35	3.162,28	11,627	58,14	+8,30
40	10.000,00	13,288	66,44	+8,30

Comparação em 10, 20, 30 e 40 dB:

SNR (dB)	C (kbit/s)	ΔC (+10 dB)	C / C(10 dB antes)	C/B (bit/s/Hz)
10	17,30	–	–	3,459
20	33,29	+15,99	1,925	6,658
30	49,84	+16,55	1,497	9,967
40	66,44	+16,60	1,333	13,288

```
================================================================================
EXPERIMENTO 3 - Influência do ruído: B = 5 kHz, SNR de 0 a 40 dB
================================================================================
SNR = 10^(SNR_dB/10)  e  C = B·log2(1 + SNR):

   SNR (dB)   SNR (linear)   log2(1+SNR)   C (kbit/s)     ΔC (+5 dB)
  ------------------------------------------------------------------
          0          1,00          1,000         5,00              -
          5          3,16          2,057        10,29          +5,29
         10         10,00          3,459        17,30          +7,01
         15         31,62          5,028        25,14          +7,84
         20        100,00          6,658        33,29          +8,15
         25        316,23          8,309        41,55          +8,26
         30     1.000,00           9,967        49,84          +8,29
         35     3.162,28          11,627        58,14          +8,30
         40    10.000,00          13,288        66,44          +8,30

Comparação em 10, 20, 30 e 40 dB:

   SNR (dB)   SNR (linear)   C (kbit/s)   ΔC (+10 dB)   C / C(-10 dB)   C/B (bit/s/Hz)
  ------------------------------------------------------------------------------------
         10             10        17,30             -               -            3,459
         20            100        33,29        +15,99           1,925            6,658
         30          1.000        49,84        +16,55           1,497            9,967
         40         10.000        66,44        +16,60           1,333           13,288

Observações calculadas:
  - +10 dB = SNR x 10; em SNR alta isso soma ≈ B·log2(10) = 16,61 kbit/s (soma, não multiplica)
  - De 10 para 40 dB a SNR é multiplicada por 1.000, mas C só por 3,84
  - Em 0 dB (sinal = ruído) C = B = 5.000 bit/s
  gráfico salvo em: graficos\exp3_ruido.png
```

Figura 13 – Saída do Experimento 3.

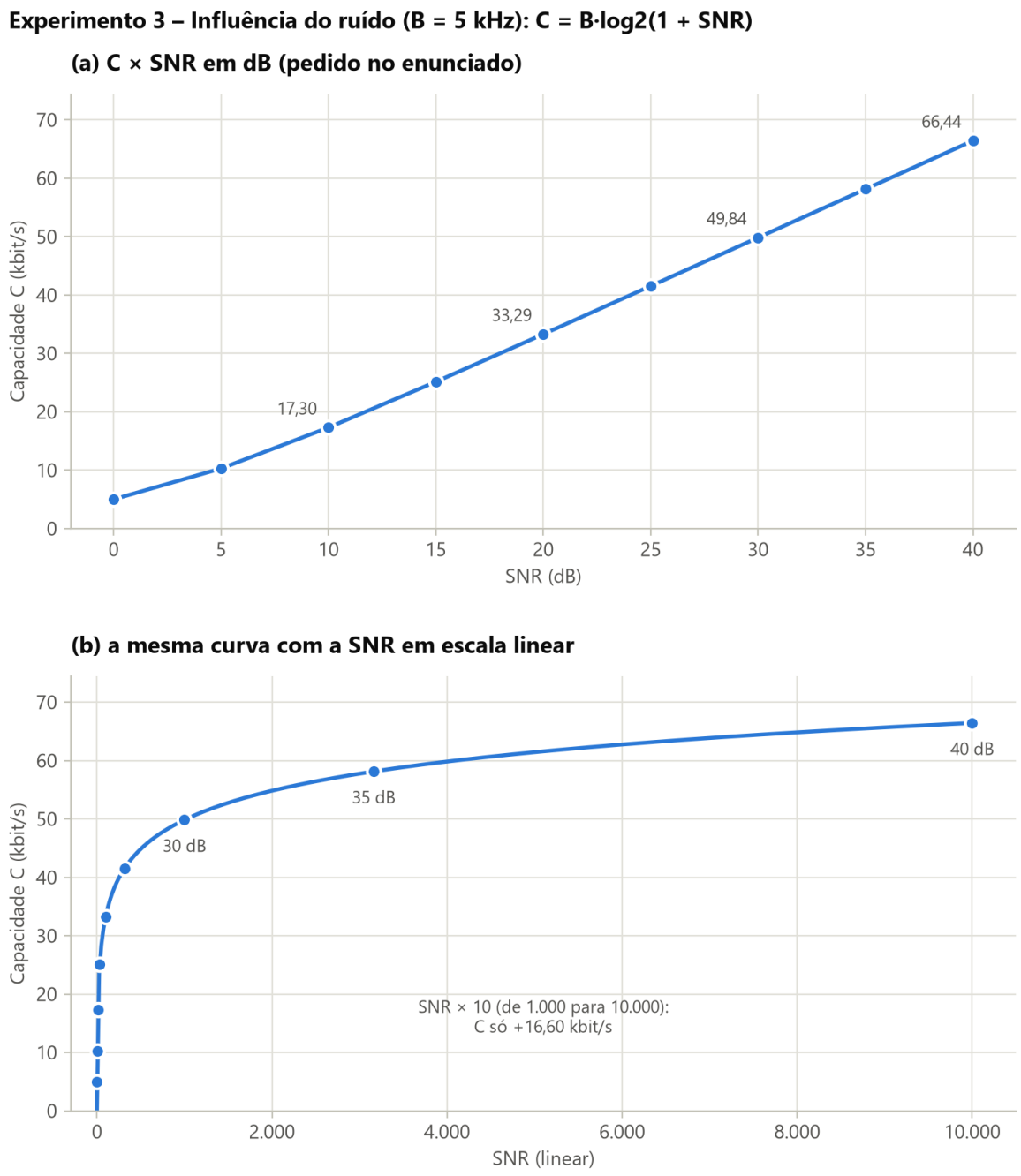


Figura 14 – Experimento 3: (a) C × SNR em dB, pedido no enunciado; (b) a mesma curva com a SNR em escala linear.

Qual é o impacto de aumentar a SNR em 10 dB?

Somar 10 dB é multiplicar a SNR por 10, e isso **soma** à capacidade um valor quase constante: +15,99, +16,55 e +16,60 kbit/s, que se aproxima de $B \cdot \log_2 10 = 16{,}61$ kbit/s (cerca de 3,32 bit/s/Hz). O ganho relativo, por outro lado, cai a cada passo: ×1,925, ×1,497 e ×1,333. De 10 para 40 dB, a SNR foi multiplicada por 1.000, e a capacidade por apenas 3,84.

Por que a relação entre SNR e capacidade não é linear?

Porque a capacidade depende de $\log_2(1 + \text{SNR})$. No gráfico (b), com a SNR em escala linear, a curva sobe rápido no início e depois quase não sai do lugar: de SNR = 1.000 para 10.000 o ganho é de só

16,60 kbit/s. No gráfico (a) a curva parece quase uma reta, porque o dB já é uma escala logarítmica. Mesmo assim, ela não é reta perto de 0 dB: os ganhos a cada 5 dB crescem de 5,29 para 8,30 kbit/s e só então se estabilizam, porque $\log_2(1 + SNR)$ só se comporta como $\log_2(SNR)$ quando a SNR é bem maior que 1.

Como o ruído limita a capacidade?

O ruído define quantos níveis do sinal o receptor ainda consegue distinguir, N_max = √(1 + SNR), e, portanto, quantos bits cada símbolo pode carregar. Em 0 dB (potência do sinal igual à do ruído), N_max = √2 ≈ 1,41 níveis e a capacidade é igual a B (5 kbit/s). Em 40 dB, o canal chega a 13,29 bit/s/Hz (66,44 kbit/s). Para qualquer SNR finita, a capacidade é finita, não importa a modulação usada: com mais ruído a SNR cai, e a capacidade cai junto.

7. Experimento 4 – Nyquist versus Shannon

B = 3 kHz, SNR = 30 dB (1.000 em escala linear) e N = 2, 4, 8, 16, 32 e 64 níveis.

- Limite de Shannon: C = 3.000 × $\log_2(1 + 1.000)$ = **29,90 kbit/s** (29.901,68 bit/s).
- Número máximo de níveis distinguíveis: N_max = √1001 = **31,64 níveis** (4,984 bits por símbolo).

N	Bits por símbolo	Nyquist (kbit/s)	Shannon (kbit/s)	Nyquist / Shannon	Situação
2	1	6,00	29,90	20,1 %	dentro do limite
4	2	12,00	29,90	40,1 %	dentro do limite
8	3	18,00	29,90	60,2 %	dentro do limite
16	4	24,00	29,90	80,3 %	dentro do limite
32	5	30,00	29,90	100,3 %	**ultrapassa (+98,32 bit/s)**
64	6	36,00	29,90	120,4 %	**ultrapassa (+6.098,32 bit/s)**

```
======================================================================
EXPERIMENTO 4 - Nyquist x Shannon: B = 3 kHz, SNR = 30 dB (1.000)
======================================================================
Limite de Shannon: C = 3.000 x log2(1 + 1.000) = 29,90 kbit/s (29.901,68 bit/s)
N_max = √(1 + SNR) = 31,64 níveis = 4,984 bits/símbolo

     N  m (bits)  Nyquist (kbit/s)  Shannon (kbit/s)  Nyquist/Shannon   Situação
  ------------------------------------------------------------------------------------
     2         1              6,00             29,90            20,1 %   dentro do limite
     4         2             12,00             29,90            40,1 %   dentro do limite
     8         3             18,00             29,90            60,2 %   dentro do limite
    16         4             24,00             29,90            80,3 %   dentro do limite
    32         5             30,00             29,90           100,3 %   ULTRAPASSA o limite (+98,32 bit/s)
    64         6             36,00             29,90           120,4 %   ULTRAPASSA o limite (+6.098,32 bit/s)

Configurações em que Nyquist ultrapassa Shannon: N = 32, 64
Obs.: o slide 24 da aula 12 arredonda log10(1.001) ≈ 3 e log10(2) ≈ 0,3 e obtém 30.000 bit/s.
      Com esse valor N = 32 daria exatamente o limite; pelo valor exato (29.901,68 bit/s),
      e porque N_max = 31,64 < 32, N = 32 já ultrapassa.
  gráfico salvo em: graficos\exp4_nyquist_x_shannon.png

Saída gravada em: saida_experimentos.txt
```

Figura 15 – Saída do Experimento 4.

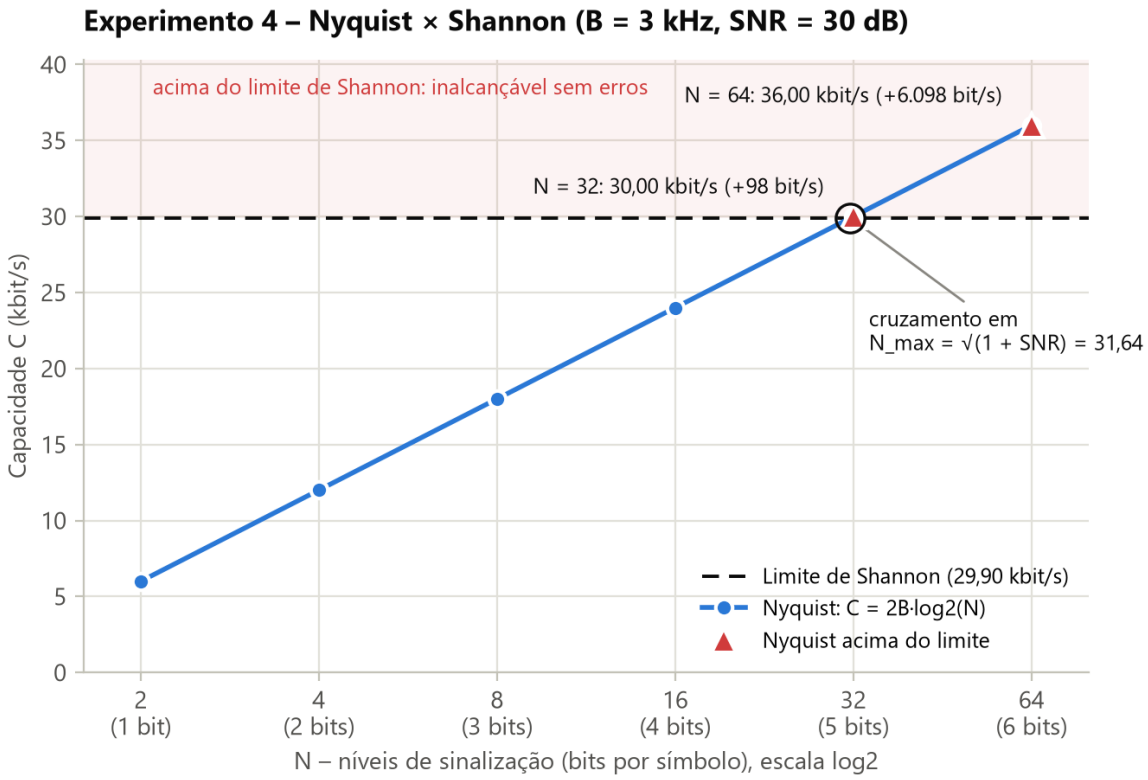


Figura 16 – Experimento 4: Nyquist (para cada N) e o limite de Shannon no mesmo gráfico.

Configurações em que a taxa de Nyquist ultrapassa o limite de Shannon: N = 32 e N = 64. Com N = 32 a diferença é de apenas 98,32 bit/s (0,33 %), e é aqui que o arredondamento do slide 24 da aula 12 faz diferença: com o valor do slide (30.000 bit/s), N = 32 daria exatamente o limite. Pelo valor exato, e também porque N_max = 31,64 é menor que 32, N = 32 já ultrapassa. No gráfico, a reta de Nyquist cruza o limite exatamente em N = N_max.

Por que aumentar N não permite superar o limite de Shannon?

Porque, com a SNR fixa, o receptor só distingue até N_max = √(1 + SNR) ≈ 31,64 níveis. Com 32 ou 64 níveis, os níveis vizinhos ficam mais próximos do que o ruído permite separar, e o receptor passa a confundir um com outro. A fórmula de Nyquist ainda devolve um número (36 kbit/s com N = 64), mas essa taxa não pode ser obtida sem erros. O teorema de Shannon diz que, acima de C, nenhuma codificação consegue tornar a probabilidade de erro tão baixa quanto se queira, então os bits que passam corretamente nunca ultrapassam os 29,90 kbit/s.

Qual a diferença entre o canal ideal e o canal com ruído?

No canal ideal (Nyquist) o único limite é a largura de banda, que fixa a taxa de símbolos em 2B (6.000 símbolos por segundo aqui). A capacidade cresce sem limite com N: 6, 12, 18, 24, 30, 36 kbit/s, e continuaria subindo. No canal com ruído (Shannon) existe um teto que depende só de B e da SNR, 29,90 kbit/s neste canal, seja qual for a modulação. Esse teto é um limite teórico superior. Na prática, codificações e modulações reais ficam abaixo dele (aula 12, slide 21).

Por que B, codificação e SNR devem ser considerados conjuntamente?

Porque cada um define uma parte da taxa. B define quantos símbolos por segundo o canal aceita (2B), a codificação ou modulação define quantos bits vão em cada símbolo ($\log_2 N$), e a SNR define quantos bits por símbolo o receptor consegue distinguir ($\log_2 N_{max} = 4{,}98$ aqui). Olhar só um deles leva a conclusões erradas. Nyquist sozinho "promete" 36 kbit/s com 64 níveis. Shannon sozinho diz o teto, mas não diz que modulação usar. Neste canal, a maior potência de 2 dentro do limite é N = 16 (24 kbit/s, 80,3 % da capacidade). Para passar de 29,90 kbit/s é preciso aumentar B, que tem efeito linear, ou a SNR, que tem efeito logarítmico. Pela tabela do Experimento 2, com B = 3 kHz, passar de 30 para 40 dB (SNR de 1.000 para 10.000) leva o limite de 29,90 para só 39,86 kbit/s, enquanto dobrar B dobra o limite. Além disso, modulações reais precisam de uma folga de SNR para manter a taxa de erro de bits baixa, o que afasta ainda mais a taxa prática do limite.

8. Uso de Inteligência Artificial

Ferramenta utilizada: Claude (Anthropic), via Claude Code.

Etapas em que ajudou: planejamento do projeto a partir de um rascunho do arquivo e planejamento (`planejamento.md`); escrita do programa `capacidade.py`, inclusive o modo de validação e os gráficos; previsão dos resultados de cada experimento, calculados à mão antes da primeira execução; e revisão deste relatório, analisando a partir das saídas reais.

Como as sugestões foram validadas:

- O programa foi executado por mim, e as tabelas, os gráficos e os prints deste relatório são dessa execução (31/10 e 01/11/2026). A saída foi idêntica à do teste feito pela IA durante o desenvolvimento.
- O modo `validar` reproduziu os exemplos numéricos da aula 12 (80 bit/s no slide 14; 30 dB = 1.000 no slide 23; o canal telefônico do slide 24) e passou nas verificações internas (9/9).
- O cálculo de Shannon foi feito por dois caminhos (`--snr-db 30` e `--snr 1000`) e deu o mesmo resultado.
- As previsões do `planejamento.md`, feitas antes de executar o programa, foram comparadas com as saídas.

Previsões confirmadas pela execução:

- Nyquist: 2, 4, 6 e 8 kbit/s para B = 1 kHz, e 5 e 10 vezes esses valores para 5 e 10 kHz.
- Shannon: menor capacidade de 9,97 kbit/s (1 kHz, SNR = 1.000) e maior de 156,1 kbit/s (10 kHz, SNR = 50.000); SNR × 5 multiplicando C por cerca de 1,23, e não por 5.
- Ruído: 17,30, 33,29, 49,84 e 66,44 kbit/s em 10, 20, 30 e 40 dB, com ganho de cerca de 16,6 kbit/s a cada 10 dB.
- Nyquist × Shannon: N = 32 e N = 64 ultrapassariam o limite de 29,90 kbit/s, e N = 32 só por cerca de 98 bit/s.

Sugestão confirmada e adaptada: ao conversar sobre o slide 24, a IA apontou que o resultado de 30.000 bit/s vem de dois arredondamentos ($\log_{10} 1001 \approx 3$ e $\log_{10} 2 \approx 0{,}3$) e que o valor exato é 29.901,68 bit/s. Isso foi confirmado pelo modo `validar`, que calcula os dois valores. A observação foi incorporada à análise do Experimento 4, porque decide se N = 32 fica "no limite" ou "acima" dele.

Sugestões corrigidas:

- A IA previu que multiplicar a SNR por 5 somaria **exatamente** $\log_2 5 = 2{,}3219$ bit/s/Hz à eficiência. O programa mediu 2,3208 (de 1.000 para 5.000) e 2,3218 (de 10.000 para 50.000). A diferença vem do "+1" da fórmula: $\log_2(1 + 5 \cdot SNR) - \log_2(1 + SNR)$ só tende a $\log_2 5$ quando a SNR é muito maior que 1. O relatório usa os valores medidos e trata $\log_2 5$ como aproximação.
- Na primeira versão do gráfico do Experimento 4, N_max era desenhado como uma linha vertical. No eixo em escala $\log_2$, essa linha ficava praticamente em cima de N = 32 (31,64 contra 32) e cortava os rótulos e a legenda. Ela foi trocada pela marcação do ponto em que a reta de Nyquist cruza o limite de Shannon, que é o próprio N_max.
- Nos testes, as mensagens de erro saíram com os acentos trocados, porque só a saída normal do programa estava em UTF-8, e não a saída de erros. Isso foi corrigido antes da execução oficial.

9. Conclusão

Os modelos de Nyquist e de Shannon foram implementados num programa que recebe os parâmetros, mostra os resultados com unidades e reproduz os exemplos da aula 12. Os quatro experimentos mostraram três comportamentos diferentes. A capacidade cresce linearmente com a largura de banda, linearmente com os bits por símbolo e apenas logaritmicamente com o número de níveis e com a SNR. Multiplicar B por 10 multiplicou a capacidade por 10, e multiplicar a SNR por 50 a multiplicou por apenas 1,57.

O Experimento 4 juntou os dois modelos. Num canal de 3 kHz com 30 dB, Nyquist permite escolher qualquer N, mas o ruído limita os níveis distinguíveis a cerca de 31,6, e a capacidade a 29,90 kbit/s. Com 32 níveis ou mais, a taxa calculada por Nyquist passa desse limite e já não pode ser obtida sem erros. A comparação também mostrou que o arredondamento usado no slide da aula (30.000 bit/s) esconde justamente esse caso.

Apêndice A – Código-fonte completo (`capacidade.py`)

```
"""
Atividade 10 - Capacidade de Canais de Comunicação (Nyquist e Shannon)
Redes de Computadores - UNISINOS 2026/2 - Matheus Milanezi

Modelos implementados:
  - Nyquist (canal ideal, sem ruído):  C = 2B·log2(N)
  - Shannon (canal com ruído):         C = B·log2(1 + SNR)
      C = capacidade (bit/s), B = largura de banda (Hz),
      N = número de níveis de sinalização, SNR = razão sinal-ruído em escala LINEAR.
  - Conversão da SNR:  SNR = 10^(SNR_dB/10)   e   SNR_dB = 10·log10(SNR)
  - Ligação entre os dois (aula 12): o ruído limita os níveis distinguíveis a
      N_max = sqrt(1 + SNR), e 2B·log2(N_max) = B·log2(1 + SNR).

Uso:
  python capacidade.py                                    os 4 experimentos (tabelas + gráficos)
  python capacidade.py experimentos [--sem-graficos] [--pasta DIR] [--saida ARQ]
  python capacidade.py nyquist  --B 3k --N 8              capacidade de Nyquist
  python capacidade.py shannon  --B 3k --snr 1000         Shannon com SNR linear
  python capacidade.py shannon  --B 3k --snr-db 30        Shannon com SNR em dB
  python capacidade.py comparar --B 3k --snr-db 30 --N 32  Nyquist x Shannon (+ N_max)
  python capacidade.py validar                            confere com os exemplos da aula 12

  B aceita os sufixos k, M e G e a unidade Hz: 3k = 3kHz = 3000 = 3000Hz.
  Números decimais podem usar ponto ou vírgula (1.5k ou 1,5k); não use separador de milhar.
"""

import argparse
import math
import re
import sys
from contextlib import ExitStack, redirect_stdout
from pathlib import Path

PASTA = Path(__file__).resolve().parent

# Parâmetros dos experimentos (enunciado da Atividade 10)
EXP1_B = [1e3, 5e3, 10e3]                     # Hz
EXP1_N = [2, 4, 8, 16]                        # níveis de sinalização
EXP2_SNR = [1_000, 5_000, 10_000, 50_000]     # escala linear
EXP2_B = [k * 1e3 for k in range(1, 11)]      # 1 a 10 kHz, passo de 1 kHz
EXP3_B = 5e3
EXP3_SNR_DB = list(range(0, 41, 5))           # 0 a 40 dB, passo de 5 dB
EXP3_COMPARAR = [10, 20, 30, 40]
EXP4_B = 3e3
EXP4_SNR_DB = 30
EXP4_N = [2, 4, 8, 16, 32, 64]

# Cores dos gráficos: séries em ordem fixa (paleta validada para daltonismo) e tintas neutras
SERIES = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100"]
MARCADORES = ["o", "s", "^", "D"]
COR = {
    "fundo": "#ffffff",
    "tinta": "#0b0b0b",       # títulos
    "tinta2": "#52514e",      # rótulos e textos
    "muda": "#898781",        # elementos de apoio
    "grade": "#e1e0d9",
    "eixo": "#c3c2b7",
    "critico": "#d03b3b",     # acima do limite de Shannon
}
LINHA = dict(linewidth=2, markersize=7, markeredgecolor=COR["fundo"], markeredgewidth=1.5)


# ---------------------------------------------------------------------------
# Núcleo: os dois modelos e a conversão da SNR
# ---------------------------------------------------------------------------

def nyquist(B: float, N: float) -> float:
    """Capacidade de um canal ideal (sem ruído), em bit/s."""
    return 2 * B * math.log2(N)


def shannon(B: float, snr: float) -> float:
    """Capacidade máxima de um canal com ruído, em bit/s (SNR em escala linear)."""
    return B * math.log2(1 + snr)


def db_para_linear(snr_db: float) -> float:
```

```
    return 10 ** (snr_db / 10)


def linear_para_db(snr: float) -> float:
    return 10 * math.log10(snr) if snr > 0 else -math.inf


def n_max(snr: float) -> float:
    """Número máximo de níveis que o receptor ainda distingue com essa SNR (aula 12)."""
    return math.sqrt(1 + snr)


# ---------------------------------------------------------------------------
# Leitura e validação dos parâmetros
# ---------------------------------------------------------------------------

SUFIXOS = {"k": 1e3, "K": 1e3, "M": 1e6, "G": 1e9}


def ler_numero(texto: str, nome: str) -> float:
    s = texto.strip()
    if re.fullmatch(r"[+-]?\d{1,3}(\.\d{3})+", s):
        raise ValueError(f"{nome} = {texto!r} é ambíguo: escreva sem separador de milhar "
                         f"(ex.: {s.replace('.', '')})")
    try:
        v = float(s.replace(",", "."))
    except ValueError:
        raise ValueError(f"{nome} inválido: {texto!r} não é um número") from None
    if not math.isfinite(v):
        raise ValueError(f"{nome} deve ser um número finito")
    return v


def ler_frequencia(texto: str) -> float:
    """'3k', '3kHz', '3000' ou '1.5M' -> valor em Hz (deve ser > 0)."""
    s = texto.strip()
    if s.lower().endswith("hz"):
        s = s[:-2].strip()
    mult = 1.0
    if s and s[-1] in SUFIXOS:
        mult = SUFIXOS[s[-1]]
        s = s[:-1]
    B = ler_numero(s, "B") * mult
    if B <= 0:
        raise ValueError(f"a largura de banda B deve ser maior que 0 Hz (recebido: {texto})")
    return B


def ler_niveis(texto: str) -> int:
    N = ler_numero(texto, "N")
    if not N.is_integer() or N < 2:
        raise ValueError(f"o número de níveis N deve ser um inteiro >= 2 (recebido: {texto})")
    return int(N)


def ler_snr(texto: str) -> float:
    snr = ler_numero(texto, "SNR")
    if snr < 0:
        raise ValueError(f"a SNR linear deve ser >= 0 (recebido: {texto}); "
                         "para informar em dB use --snr-db")
    return snr


def ler_snr_db(texto: str) -> float:
    try:
        return db_para_linear(ler_numero(texto, "SNR (dB)"))
    except OverflowError:
        raise ValueError(f"SNR = {texto} dB é grande demais para a escala linear") from None


# ---------------------------------------------------------------------------
# Exibição com unidades (padrão brasileiro: 29.901,68)
# ---------------------------------------------------------------------------

def br(x: float, casas: int = 2) -> str:
    return f"{x:,.{casas}f}".replace(",", "_").replace(".", ",").replace("_", ".")


def br_curto(x: float, casas: int = 3) -> str:
```

```
    """Como br(), mas sem zeros à direita: 3000 -> '3.000', 1.5 -> '1,5'."""
    s = br(x, casas)
    return s.rstrip("0").rstrip(",") if "," in s else s


def br_cient(x: float, casas: int = 3) -> str:
    mant, exp = f"{x:.{casas}e}".split("e")
    return f"{mant.replace('.', ',')} × 10^{int(exp)}"


def taxa(c: float) -> str:
    """Capacidade na unidade mais legível e também em bit/s."""
    for fator, unidade in ((1e9, "Gbit/s"), (1e6, "Mbit/s"), (1e3, "kbit/s")):
        if abs(c) >= fator:
            return f"{br(c / fator)} {unidade} ({br(c)} bit/s)"
    return f"{br(c)} bit/s"


def frequencia(B: float) -> str:
    for fator, unidade in ((1e9, "GHz"), (1e6, "MHz"), (1e3, "kHz")):
        if B >= fator:
            return f"{br_curto(B / fator)} {unidade} ({br_curto(B)} Hz)"
    return f"{br_curto(B)} Hz"


def snr_texto(snr: float) -> str:
    db = linear_para_db(snr)
    return f"{br_curto(snr)} (linear) = {'-∞' if db == -math.inf else br(db)} dB"


def linha(titulo: str) -> None:
    print("=" * 78)
    print(titulo)
    print("=" * 78)


def mostrar_caminho(caminho: Path) -> str:
    try:
        return str(caminho.relative_to(Path.cwd()))
    except ValueError:
        return str(caminho)


# ---------------------------------------------------------------------------
# Modo parâmetro: nyquist, shannon e comparar
# ---------------------------------------------------------------------------

def mostrar_snr(snr: float, snr_db: str | None) -> None:
    if snr_db is not None:
        print(f"  SNR = 10^({snr_db}/10) = {snr_texto(snr)}")
    else:
        print(f"  SNR = {snr_texto(snr)}")


def modo_nyquist(B: float, N: int) -> None:
    linha("Nyquist - canal ideal (sem ruído): C = 2B·log2(N)")
    m = math.log2(N)
    print(f"  B = {frequencia(B)}")
    print(f"  N = {N} níveis  ->  m = log2({N}) = {br_curto(m)} bits/símbolo")
    if not m.is_integer():
        print("      (N não é potência de 2: m fracionário, vale como limite teórico)")
    print(f"  taxa de sinalização = 2B = {br_curto(2 * B)} baud (símbolos/s)")
    print(f"  C = 2 × {br_curto(B)} × {br_curto(m)} = {taxa(nyquist(B, N))}")


def modo_shannon(B: float, snr: float, snr_db: str | None) -> None:
    linha("Shannon - canal com ruído: C = B·log2(1 + SNR)")
    efic = math.log2(1 + snr)
    print(f"  B = {frequencia(B)}")
    mostrar_snr(snr, snr_db)
    print(f"  log2(1 + SNR) = log2({br_curto(1 + snr)}) = {br(efic, 4)} bit/s/Hz"
          "  (eficiência espectral máxima)")
    print(f"  C = {br_curto(B)} × {br(efic, 4)} = {taxa(shannon(B, snr))}")
    nm = n_max(snr)
    print(f"  N_max = √(1 + SNR) = {br(nm)} níveis ({br(math.log2(nm), 3)} bits/símbolo)")


def modo_comparar(B: float, snr: float, snr_db: str | None, N: int) -> None:
    linha("Nyquist (canal ideal) x Shannon (canal com ruído)")
```

```
    cn, cs, nm = nyquist(B, N), shannon(B, snr), n_max(snr)
    print(f"  B = {frequencia(B)}")
    mostrar_snr(snr, snr_db)
    print(f"  N = {N} níveis ({br_curto(math.log2(N))} bits/símbolo)")
    print()
    print(f"  Nyquist  C = 2B·log2(N)      = {taxa(cn)}")
    print(f"  Shannon  C = B·log2(1 + SNR) = {taxa(cs)}")
    print(f"  N_max = √(1 + SNR)           = {br(nm)} níveis ({br(math.log2(nm), 3)} bits/símbolo)")
    print()
    if cs == 0:
        print("  Com SNR = 0 a capacidade de Shannon é 0: nenhuma taxa é alcançável.")
    elif cn > cs:
        print(f"  Resultado: N = {N} > N_max -> a taxa de Nyquist ULTRAPASSA o limite de Shannon")
        print(f"             em {taxa(cn - cs)} (+{br((cn / cs - 1) * 100)} %).")
        print("            Essa taxa não pode ser obtida com erro arbitrariamente baixo neste canal:")
        print(f"             o ruído não deixa o receptor distinguir {N} níveis.")
    else:
        print(f"  Resultado: N = {N} <= N_max -> a taxa de Nyquist fica dentro do limite de Shannon")
        print(f"             (usa {br(cn / cs * 100)} % da capacidade do canal).")


# ---------------------------------------------------------------------------
# Experimentos da atividade
# ---------------------------------------------------------------------------

def preparar_matplotlib():
    """Importa o matplotlib só quando os gráficos são pedidos e aplica o estilo."""
    import matplotlib
    matplotlib.use("Agg")                        # gera os PNGs sem abrir janelas
    import matplotlib.pyplot as plt
    from matplotlib import font_manager

    instaladas = {f.name for f in font_manager.fontManager.ttflist}
    fontes = [f for f in ("Segoe UI", "DejaVu Sans") if f in instaladas] or ["sans-serif"]
    plt.rcParams.update({
        "font.family": fontes,
        "font.size": 10,
        "figure.facecolor": COR["fundo"],
        "axes.facecolor": COR["fundo"],
        "axes.edgecolor": COR["eixo"],
        "axes.labelcolor": COR["tinta2"],
        "axes.titlecolor": COR["tinta"],
        "axes.titlesize": 12,
        "axes.titleweight": "bold",
        "axes.titlelocation": "left",
        "axes.titlepad": 12,
        "axes.spines.top": False,
        "axes.spines.right": False,
        "axes.grid": True,
        "axes.axisbelow": True,
        "grid.color": COR["grade"],
        "grid.linewidth": 0.8,
        "xtick.color": COR["eixo"],
        "ytick.color": COR["eixo"],
        "xtick.labelcolor": COR["tinta2"],
        "ytick.labelcolor": COR["tinta2"],
        "legend.frameon": False,
        "legend.fontsize": 9,
        "savefig.dpi": 200,
    })
    return plt


def formato_br(ax, eixo: str = "y") -> None:
    from matplotlib.ticker import FuncFormatter
    fmt = FuncFormatter(lambda v, _: br_curto(v))
    (ax.yaxis if eixo == "y" else ax.xaxis).set_major_formatter(fmt)


def rotulo(ax, texto: str, xy: tuple, dx: float = 8, dy: float = 0, **kw) -> None:
    """Rótulo direto (em tinta neutra) ao lado de um ponto; pode ficar fora da área do gráfico."""
    opcoes = dict(textcoords="offset points", va="center", color=COR["tinta2"],
                  fontsize=9, annotation_clip=False)
    opcoes.update(kw)
    ax.annotate(texto, xy=xy, xytext=(dx, dy), **opcoes)


def salvar(plt, fig, pasta: Path, nome: str) -> None:
    pasta.mkdir(parents=True, exist_ok=True)
```

```
    caminho = pasta / nome
    fig.savefig(caminho, bbox_inches="tight", pad_inches=0.2)
    plt.close(fig)
    print(f"  gráfico salvo em: {mostrar_caminho(caminho)}")


def experimento_1(plt, pasta: Path) -> None:
    linha("EXPERIMENTO 1 - Capacidade de Nyquist (canal ideal): C = 2B·log2(N)")
    C = {B: [nyquist(B, N) for N in EXP1_N] for B in EXP1_B}

    print("Capacidade C em kbit/s:\n")
    rot = 28
    print(f"  {'N (níveis)':<{rot}}" + "".join(f"{N:>10}" for N in EXP1_N))
    print(f"  {'m = log2 N (bits/símbolo)':<{rot}}"
          + "".join(f"{br_curto(math.log2(N)):>10}" for N in EXP1_N))
    print("  " + "-" * (rot + 10 * len(EXP1_N)))
    for B in EXP1_B:
        print(f"  {'B = ' + br_curto(B / 1e3) + ' kHz':<{rot}}"
              + "".join(f"{br(c / 1e3):>10}" for c in C[B]))

    print("\nObservações calculadas:")
    b0, b1 = EXP1_B[0], EXP1_B[-1]
    razoes = sorted({round(c1 / c0, 9) for c0, c1 in zip(C[b0], C[b1])})
    print(f"  - B × {br_curto(b1 / b0)} ({br_curto(b0 / 1e3)} -> {br_curto(b1 / 1e3)} kHz): "
          f"C × {' / '.join(br(r) for r in razoes)} para todo N -> C é proporcional a B")
    for B in EXP1_B:
        difs = sorted({round(C[B][i + 1] - C[B][i], 6) for i in range(len(EXP1_N) - 1)})
        print(f"  - B = {br_curto(B / 1e3)} kHz: cada vez que N dobra (+1 bit/símbolo), C soma "
              f"{' / '.join(br(d / 1e3) for d in difs)} kbit/s (= 2B)")
    n0, n1 = EXP1_N[0], EXP1_N[-1]
    print(f"  - N × {n1 // n0} ({n0} -> {n1} níveis) multiplica C por apenas "
          f"{br(C[b0][-1] / C[b0][0])} (= log2 {n1} / log2 {n0})")

    if plt:
        fig, ax = plt.subplots(figsize=(7.5, 4.5))
        for k, B in enumerate(EXP1_B):
            y = [c / 1e3 for c in C[B]]
            nome = f"B = {br_curto(B / 1e3)} kHz"
            ax.plot(EXP1_N, y, color=SERIES[k], marker=MARCADORES[k], label=nome, **LINHA)
            rotulo(ax, f"{nome}: {br_curto(y[-1])} kbit/s", (EXP1_N[-1], y[-1]))
        ax.set_xticks(EXP1_N, [f"{N}\n({br_curto(math.log2(N))} bit{'s' if N > 2 else ''})"
                               for N in EXP1_N])
        ax.set_xlim(1, 17)
        ax.set_ylim(0, max(C[b1]) / 1e3 * 1.1)
        formato_br(ax)
        ax.set_title("Experimento 1 – Capacidade de Nyquist: C = 2B·log2(N)")
        ax.set_xlabel("N – níveis de sinalização (bits por símbolo)")
        ax.set_ylabel("Capacidade C (kbit/s)")
        ax.legend(loc="upper left")
        salvar(plt, fig, pasta, "exp1_nyquist.png")
    print()


def experimento_2(plt, pasta: Path) -> None:
    linha("EXPERIMENTO 2 - Capacidade de Shannon: C = B·log2(1 + SNR)")
    C = {snr: [shannon(B, snr) for B in EXP2_B] for snr in EXP2_SNR}

    print("Capacidade C em kbit/s:\n")
    rot, col = 28, 12
    print(f"  {'SNR (linear)':<{rot}}" + "".join(f"{br_curto(s):>{col}}" for s in EXP2_SNR))
    print(f"  {'SNR (dB)':<{rot}}" + "".join(f"{br(linear_para_db(s)):>{col}}" for s in EXP2_SNR))
    print(f"  {'log2(1 + SNR) (bit/s/Hz)':<{rot}}"
          + "".join(f"{br(math.log2(1 + s), 3):>{col}}" for s in EXP2_SNR))
    print("  " + "-" * (rot + col * len(EXP2_SNR)))
    for i, B in enumerate(EXP2_B):
        print(f"  {'B = ' + br_curto(B / 1e3) + ' kHz':<{rot}}"
              + "".join(f"{br(C[s][i] / 1e3):>{col}}" for s in EXP2_SNR))

    todos = [(C[s][i], B, s) for s in EXP2_SNR for i, B in enumerate(EXP2_B)]
    maior, menor = max(todos), min(todos)
    print()
    print(f"Maior capacidade: {taxa(maior[0])}  (B = {br_curto(maior[1] / 1e3)} kHz, "
          f"SNR = {br_curto(maior[2])})")
    print(f"Menor capacidade: {taxa(menor[0])}  (B = {br_curto(menor[1] / 1e3)} kHz, "
          f"SNR = {br_curto(menor[2])})")
    print(f"Maior / menor = {br(maior[0] / menor[0])}")

    print("\nObservações calculadas:")
    razoes = sorted({round(C[s][-1] / C[s][0], 9) for s in EXP2_SNR})
```

```
    print(f"  - Para cada SNR, C/B é constante: C(10 kHz) / C(1 kHz) = "
          f"{' / '.join(br(r) for r in razoes)} -> C cresce linearmente com B")
    for s0, s1 in [(1_000, 5_000), (10_000, 50_000), (1_000, 50_000)]:
        e0, e1 = math.log2(1 + s0), math.log2(1 + s1)
        print(f"  - SNR × {br_curto(s1 / s0)} ({br_curto(s0)} -> {br_curto(s1)}): C × {br(e1 / e0, 4)}"
              f" para todo B; a eficiência sobe só {br(e1 - e0, 4)} bit/s/Hz")
    e0 = math.log2(1 + EXP2_SNR[0])
    snr5 = (1 + EXP2_SNR[0]) ** 5 - 1
    print(f"  - Para C × 5 a partir de SNR = {br_curto(EXP2_SNR[0])} seria preciso "
          f"(1 + SNR)^5 - 1 = {br_cient(snr5)} ({br(linear_para_db(snr5), 1)} dB)")

    if plt:
        fig, ax = plt.subplots(figsize=(7.5, 4.5))
        x = [B / 1e3 for B in EXP2_B]
        for k, s in enumerate(EXP2_SNR):
            y = [c / 1e3 for c in C[s]]
            ax.plot(x, y, color=SERIES[k], marker=MARCADORES[k], markersize=6,
                    label=f"SNR = {br_curto(s)} ({br(linear_para_db(s), 1)} dB)",
                    **{c: v for c, v in LINHA.items() if c != "markersize"})
            rotulo(ax, f"SNR = {br_curto(s)}", (x[-1], y[-1]))
        for valor, B, _s, texto, dx, dy, ha in [
                (*maior, f"maior: {br(maior[0] / 1e3)} kbit/s", -12, 10, "right"),
                (*menor, f"menor: {br(menor[0] / 1e3)} kbit/s", 10, -12, "left")]:
            ax.plot(B / 1e3, valor / 1e3, marker="o", markersize=12, markerfacecolor="none",
                    markeredgecolor=COR["tinta"], markeredgewidth=1.2, linestyle="none", zorder=5)
            rotulo(ax, texto, (B / 1e3, valor / 1e3), dx, dy, ha=ha, color=COR["tinta"])
        ax.set_xticks(x)
        ax.set_xlim(0.5, 10.5)
        ax.set_ylim(0, maior[0] / 1e3 * 1.1)
        formato_br(ax)
        ax.set_title("Experimento 2 – Capacidade de Shannon: C = B·log2(1 + SNR)")
        ax.set_xlabel("Largura de banda B (kHz)")
        ax.set_ylabel("Capacidade C (kbit/s)")
        ax.legend(loc="upper left")
        salvar(plt, fig, pasta, "exp2_shannon.png")
    print()


def experimento_3(plt, pasta: Path) -> None:
    linha(f"EXPERIMENTO 3 - Influência do ruído: B = {br_curto(EXP3_B / 1e3)} kHz, "
          f"SNR de {EXP3_SNR_DB[0]} a {EXP3_SNR_DB[-1]} dB")
    snrs = [db_para_linear(db) for db in EXP3_SNR_DB]
    C = [shannon(EXP3_B, s) for s in snrs]
    passo = EXP3_SNR_DB[1] - EXP3_SNR_DB[0]

    print("SNR = 10^(SNR_dB/10)  e  C = B·log2(1 + SNR):\n")
    print(f"  {'SNR (dB)':>9}{'SNR (linear)':>15}{'log2(1+SNR)':>14}{'C (kbit/s)':>13}"
          f"{'ΔC (+' + str(passo) + ' dB)':>15}")
    print("  " + "-" * 66)
    for i, db in enumerate(EXP3_SNR_DB):
        dc = "-" if i == 0 else "+" + br((C[i] - C[i - 1]) / 1e3)
        print(f"  {db:>9}{br(snrs[i]):>15}{br(math.log2(1 + snrs[i]), 3):>14}"
              f"{br(C[i] / 1e3):>13}{dc:>15}")

    print(f"\nComparação em {', '.join(str(d) for d in EXP3_COMPARAR[:-1])} e "
          f"{EXP3_COMPARAR[-1]} dB:\n")
    print(f"  {'SNR (dB)':>9}{'SNR (linear)':>15}{'C (kbit/s)':>13}{'ΔC (+10 dB)':>14}"
          f"{'C / C(-10 dB)':>16}{'C/B (bit/s/Hz)':>17}")
    print("  " + "-" * 84)
    anterior = None
    for db in EXP3_COMPARAR:
        c = C[EXP3_SNR_DB.index(db)]
        dc = "-" if anterior is None else "+" + br((c - anterior) / 1e3)
        rz = "-" if anterior is None else br(c / anterior, 3)
        print(f"  {db:>9}{br_curto(db_para_linear(db)):>15}{br(c / 1e3):>13}{dc:>14}{rz:>16}"
              f"{br(c / EXP3_B, 3):>17}")
        anterior = c
    c_ini, c_fim = C[EXP3_SNR_DB.index(EXP3_COMPARAR[0])], C[EXP3_SNR_DB.index(EXP3_COMPARAR[-1])]
    print("\nObservações calculadas:")
    print(f"  - +10 dB = SNR × 10; em SNR alta isso soma ≈ B·log2(10) = "
          f"{br(EXP3_B * math.log2(10) / 1e3)} kbit/s (soma, não multiplica)")
    print(f"  - De {EXP3_COMPARAR[0]} para {EXP3_COMPARAR[-1]} dB a SNR é multiplicada por "
          f"{br_curto(db_para_linear(EXP3_COMPARAR[-1] - EXP3_COMPARAR[0]))}, "
          f"mas C só por {br(c_fim / c_ini)}")
    print(f"  - Em 0 dB (sinal = ruído) C = B = {br_curto(C[0])} bit/s")

    if plt:
        fig, (a1, a2) = plt.subplots(2, 1, figsize=(7.5, 8.6))
        y = [c / 1e3 for c in C]
```

```
        a1.plot(EXP3_SNR_DB, y, color=SERIES[0], marker="o", **LINHA)
        for db in EXP3_COMPARAR:
            i = EXP3_SNR_DB.index(db)
            rotulo(a1, f"{br(y[i])}", (db, y[i]), -6, 9, ha="right")
        a1.set_xticks(EXP3_SNR_DB)
        a1.set_ylim(0, max(y) * 1.12)
        formato_br(a1)
        a1.set_title("(a) C × SNR em dB (pedido no enunciado)")
        a1.set_xlabel("SNR (dB)")
        a1.set_ylabel("Capacidade C (kbit/s)")

        smax = snrs[-1]
        xs = [smax * k / 400 for k in range(401)]
        a2.plot(xs, [shannon(EXP3_B, s) / 1e3 for s in xs], color=SERIES[0], linewidth=2)
        a2.plot(snrs, y, color=SERIES[0], marker="o", linestyle="none",
                **{c: v for c, v in LINHA.items() if c != "linewidth"})
        for db in (30, 35, 40):
            i = EXP3_SNR_DB.index(db)
            rotulo(a2, f"{db} dB", (snrs[i], y[i]), 0, -14, ha="center")
        i30, i40 = EXP3_SNR_DB.index(30), EXP3_SNR_DB.index(40)
        a2.text(0.5, 0.18,
                f"SNR × 10 (de {br_curto(snrs[i30])} para {br_curto(snrs[i40])}):\n"
                f"C só +{br(y[i40] - y[i30])} kbit/s",
                transform=a2.transAxes, ha="center", color=COR["tinta2"], fontsize=9)
        a2.set_xlim(-smax * 0.03, smax * 1.05)
        a2.set_ylim(0, max(y) * 1.12)
        formato_br(a2)
        formato_br(a2, "x")
        a2.set_title("(b) a mesma curva com a SNR em escala linear")
        a2.set_xlabel("SNR (linear)")
        a2.set_ylabel("Capacidade C (kbit/s)")
        fig.suptitle(f"Experimento 3 – Influência do ruído (B = {br_curto(EXP3_B / 1e3)} kHz): "
                     "C = B·log2(1 + SNR)", x=0.02, ha="left", fontsize=12, fontweight="bold",
                     color=COR["tinta"])
        fig.tight_layout(h_pad=2.5)
        salvar(plt, fig, pasta, "exp3_ruido.png")
    print()


def experimento_4(plt, pasta: Path) -> None:
    snr = db_para_linear(EXP4_SNR_DB)
    lim, nm = shannon(EXP4_B, snr), n_max(snr)
    linha(f"EXPERIMENTO 4 - Nyquist x Shannon: B = {br_curto(EXP4_B / 1e3)} kHz, "
          f"SNR = {EXP4_SNR_DB} dB ({br_curto(snr)})")
    print(f"Limite de Shannon: C = {br_curto(EXP4_B)} × log2(1 + {br_curto(snr)}) = {taxa(lim)}")
    print(f"N_max = √(1 + SNR) = {br(nm)} níveis = {br(math.log2(nm), 3)} bits/símbolo\n")

    print(f"  {'N':>4}{'m (bits)':>10}{'Nyquist (kbit/s)':>18}{'Shannon (kbit/s)':>18}"
          f"{'Nyquist/Shannon':>17}   Situação")
    print("  " + "-" * 92)
    acima = []
    for N in EXP4_N:
        c = nyquist(EXP4_B, N)
        if c > lim:
            acima.append(N)
            sit = f"ULTRAPASSA o limite (+{br(c - lim)} bit/s)"
        else:
            sit = "dentro do limite"
        print(f"  {N:>4}{br_curto(math.log2(N)):>10}{br(c / 1e3):>18}{br(lim / 1e3):>18}"
              f"{br(c / lim * 100, 1) + ' %':>17}   {sit}")
    print()
    print(f"Configurações em que Nyquist ultrapassa Shannon: "
          f"N = {', '.join(str(n) for n in acima) if acima else 'nenhuma'}")
    slide = EXP4_B * round(math.log10(1 + snr), 1) / round(math.log10(2), 1)
    print(f"Obs.: o slide 24 da aula 12 arredonda log10(1.001) ≈ 3 e log10(2) ≈ 0,3 e obtém "
          f"{br_curto(slide)} bit/s.")
    print(f"      Com esse valor N = 32 daria exatamente o limite; pelo valor exato "
          f"({br(lim)} bit/s),")
    print(f"      e porque N_max = {br(nm)} < 32, N = 32 já ultrapassa.")

    if plt:
        fig, ax = plt.subplots(figsize=(7.5, 4.6))
        from matplotlib.ticker import NullLocator
        y = [nyquist(EXP4_B, N) / 1e3 for N in EXP4_N]
        topo = max(y) * 1.12
        ax.axhspan(lim / 1e3, topo, color=COR["critico"], alpha=0.06, linewidth=0)
        ax.text(EXP4_N[0], topo * 0.97, "  acima do limite de Shannon: inalcançável sem erros",
                va="top", color=COR["critico"], fontsize=9)
        ax.axhline(lim / 1e3, color=COR["tinta"], linestyle=(0, (5, 3)), linewidth=1.4,
```

```
                label=f"Limite de Shannon ({br(lim / 1e3)} kbit/s)")
        ax.plot(EXP4_N, y, color=SERIES[0], marker="o", label="Nyquist: C = 2B·log2(N)", **LINHA)
        # a reta de Nyquist cruza o limite exatamente em N = N_max
        ax.plot(nm, lim / 1e3, marker="o", markersize=12, markerfacecolor="none",
                markeredgecolor=COR["tinta"], markeredgewidth=1.2, linestyle="none", zorder=5)
        ax.annotate(f"cruzamento em\nN_max = √(1 + SNR) = {br(nm)}", xy=(nm, lim / 1e3),
                    xytext=(34, lim / 1e3 - 6), ha="left", va="top", color=COR["tinta"],
                    fontsize=9, arrowprops=dict(arrowstyle="-", color=COR["muda"], linewidth=1))
        xa = [N for N in EXP4_N if N in acima]
        ya = [nyquist(EXP4_B, N) / 1e3 for N in xa]
        ax.plot(xa, ya, marker="^", markersize=10, color=COR["critico"],
                markeredgecolor=COR["fundo"], markeredgewidth=1.5, linestyle="none", zorder=4,
                label="Nyquist acima do limite")
        for N, v in zip(xa, ya):
            rotulo(ax, f"N = {N}: {br(v)} kbit/s (+{br_curto(v * 1e3 - lim, 0)} bit/s)",
                   (N, v), -12, 12, ha="right", color=COR["tinta"])
        ax.set_xscale("log", base=2)
        ax.set_xticks(EXP4_N, [f"{N}\n({br_curto(math.log2(N))} bit{'s' if N > 2 else ''})"
                               for N in EXP4_N])
        ax.xaxis.set_minor_locator(NullLocator())
        ax.set_xlim(EXP4_N[0] / 1.25, EXP4_N[-1] * 1.25)
        ax.set_ylim(0, topo)
        formato_br(ax)
        ax.set_title(f"Experimento 4 – Nyquist × Shannon (B = {br_curto(EXP4_B / 1e3)} kHz, "
                     f"SNR = {EXP4_SNR_DB} dB)")
        ax.set_xlabel("N – níveis de sinalização (bits por símbolo), escala log2")
        ax.set_ylabel("Capacidade C (kbit/s)")
        ax.legend(loc="lower right")
        salvar(plt, fig, pasta, "exp4_nyquist_x_shannon.png")
    print()


class Tee:
    """Escreve no terminal e num arquivo ao mesmo tempo."""

    def __init__(self, *fluxos):
        self.fluxos = fluxos

    def write(self, texto: str) -> int:
        for f in self.fluxos:
            f.write(texto)
        return len(texto)

    def flush(self) -> None:
        for f in self.fluxos:
            f.flush()


def modo_experimentos(sem_graficos: bool, pasta: Path, saida: str) -> None:
    plt = None if sem_graficos else preparar_matplotlib()
    with ExitStack() as pilha:
        if saida:
            Path(saida).parent.mkdir(parents=True, exist_ok=True)
            arq = pilha.enter_context(open(saida, "w", encoding="utf-8"))
            pilha.enter_context(redirect_stdout(Tee(sys.stdout, arq)))
        experimento_1(plt, pasta)
        experimento_2(plt, pasta)
        experimento_3(plt, pasta)
        experimento_4(plt, pasta)
    if saida:
        print(f"Saída gravada em: {mostrar_caminho(Path(saida).resolve())}")


# ---------------------------------------------------------------------------
# Validação: exemplos da aula 12 e conferências internas
# ---------------------------------------------------------------------------

def modo_validar() -> bool:
    linha("VALIDAÇÃO - exemplos da aula 12 e conferências internas")
    resultados = []

    def caso(descricao: str, ok: bool, detalhe: str) -> None:
        resultados.append(ok)
        print(f"  [{'OK' if ok else 'FALHOU'}] {descricao}")
        print(f"         {detalhe}")

    def perto(a: float, b: float) -> bool:
        return math.isclose(a, b, rel_tol=1e-12, abs_tol=1e-9)
```

```
    c = nyquist(20, 4)
    caso("Aula 12, slide 14: Nyquist com B = 20 Hz e N = 4", perto(c, 80),
         f"obtido {taxa(c)}, esperado 80 bit/s")

    s = db_para_linear(30)
    caso("Aula 12, slide 23: 30 dB em escala linear", perto(s, 1000),
         f"obtido 10^(30/10) = {br_curto(s)}, esperado 1.000")

    c = shannon(3000, 1000)
    pelo_log10 = 3000 * math.log10(1001) / math.log10(2)
    caso("Aula 12, slide 24: Shannon com B = 3.000 Hz e SNR = 1.000 (valor exato)",
         perto(c, pelo_log10),
         f"obtido {taxa(c)}\n"
         f"         pelo caminho do slide: 3.000 × log10(1.001)/log10(2) = {br(pelo_log10)} bit/s")
    slide = 3000 * round(math.log10(1001), 1) / round(math.log10(2), 1)
    caso("Aula 12, slide 24: valor do slide com os arredondamentos log10(1.001) ≈ 3, log10(2) ≈ 0,3",
         perto(slide, 30_000),
         f"3.000 × 3/0,3 = {br_curto(slide)} bit/s; diferença para o exato = "
         f"{br(slide - c)} bit/s ({br((slide / c - 1) * 100)} %)")

    dbs = [0, 3, 10, 30, 47]
    ok = sum(perto(linear_para_db(db_para_linear(d)), d) for d in dbs)
    caso("Conversão dB -> linear -> dB volta ao valor original", ok == len(dbs),
         f"{ok}/{len(dbs)} valores ({', '.join(str(d) for d in dbs)} dB)")

    casos = [(B, s) for B in EXP1_B for s in [1, 1000, 5000, 50000]]
    ok = sum(perto(nyquist(B, n_max(s)), shannon(B, s)) for B, s in casos)
    caso("Identidade da aula: 2B·log2(√(1 + SNR)) = B·log2(1 + SNR)", ok == len(casos),
         f"{ok}/{len(casos)} combinações de B e SNR")

    casos = [(B, m) for B in EXP1_B for m in range(1, 7)]
    ok = sum(perto(nyquist(B, 2 ** m), 2 * B * m) for B, m in casos)
    caso("Nyquist com N = 2^m é exatamente 2B·m", ok == len(casos),
         f"{ok}/{len(casos)} combinações (m = 1 a 6)")

    caso("Shannon nos extremos: SNR = 0 -> C = 0 e SNR = 0 dB -> C = B",
         shannon(5000, 0) == 0 and perto(shannon(5000, db_para_linear(0)), 5000),
         f"C(SNR = 0) = {br(shannon(5000, 0))} bit/s; "
         f"C(0 dB) = {br(shannon(5000, db_para_linear(0)))} bit/s com B = 5.000 Hz")

    invalidas = [(ler_niveis, "1"), (ler_niveis, "2.5"), (ler_frequencia, "-5"),
                 (ler_frequencia, "0"), (ler_snr, "-3"), (ler_snr, "1.000"), (ler_snr, "abc")]
    rejeitadas = 0
    for funcao, entrada in invalidas:
        try:
            funcao(entrada)
        except ValueError:
            rejeitadas += 1
    caso("Entradas inválidas são rejeitadas", rejeitadas == len(invalidas),
         f"{rejeitadas}/{len(invalidas)}: N = 1, N = 2.5, B = -5, B = 0, SNR = -3, "
         "SNR = '1.000' (ambíguo), SNR = 'abc'")

    print()
    print(f"Resultado: {sum(resultados)}/{len(resultados)} verificações OK")
    return all(resultados)


# ---------------------------------------------------------------------------
# Linha de comando
# ---------------------------------------------------------------------------

def main() -> None:
    for fluxo in (sys.stdout, sys.stderr):
        try:
            fluxo.reconfigure(encoding="utf-8")
        except (AttributeError, ValueError):
            pass

    ap = argparse.ArgumentParser(description="Capacidade de canais: Nyquist e Shannon")
    sub = ap.add_subparsers(dest="modo", metavar="MODO")

    p = sub.add_parser("experimentos", help="roda os 4 experimentos da atividade (padrão)")
    p.add_argument("--sem-graficos", action="store_true", help="só as tabelas, sem gerar PNGs")
    p.add_argument("--pasta", default=str(PASTA / "graficos"),
                   help="pasta dos gráficos (padrão: graficos/ ao lado do programa)")
    p.add_argument("--saida", default=str(PASTA / "saida_experimentos.txt"),
                   help="arquivo onde a saída também é gravada ('' para não gravar)")

    p = sub.add_parser("nyquist", help="C = 2B·log2(N)")
```

```
    p.add_argument("--B", required=True, help="largura de banda (Hz; aceita k/M/G: 3k)")
    p.add_argument("--N", required=True, help="número de níveis de sinalização (inteiro >= 2)")

    for nome, ajuda in (("shannon", "C = B·log2(1 + SNR)"),
                        ("comparar", "Nyquist x Shannon com os mesmos B e SNR")):
        p = sub.add_parser(nome, help=ajuda)
        p.add_argument("--B", required=True, help="largura de banda (Hz; aceita k/M/G: 3k)")
        g = p.add_mutually_exclusive_group(required=True)
        g.add_argument("--snr", help="SNR em escala linear (ex.: 1000)")
        g.add_argument("--snr-db", help="SNR em dB (ex.: 30)")
        if nome == "comparar":
            p.add_argument("--N", required=True, help="número de níveis de sinalização")

    sub.add_parser("validar", help="confere o programa com os exemplos da aula 12")
    args = ap.parse_args()

    try:
        if args.modo == "nyquist":
            modo_nyquist(ler_frequencia(args.B), ler_niveis(args.N))
        elif args.modo in ("shannon", "comparar"):
            B = ler_frequencia(args.B)
            snr = ler_snr(args.snr) if args.snr is not None else ler_snr_db(args.snr_db)
            if args.modo == "shannon":
                modo_shannon(B, snr, args.snr_db)
            else:
                modo_comparar(B, snr, args.snr_db, ler_niveis(args.N))
        elif args.modo == "validar":
            sys.exit(0 if modo_validar() else 1)
        else:
            modo_experimentos(getattr(args, "sem_graficos", False),
                              Path(getattr(args, "pasta", PASTA / "graficos")),
                              getattr(args, "saida", str(PASTA / "saida_experimentos.txt")))
    except (ValueError, OSError) as e:
        print(f"Erro: {e}", file=sys.stderr)
        sys.exit(1)


if __name__ == "__main__":
    main()
```